

Organisation des données côté client

Projet : SAE BUT2 S3 2026-2027 **Thème :** Langages de programmation **Responsables :** Usman + Omar **Partie :** Client Java + JavaFX

Sommaire

1. [Objectif](#)
 2. [Vue d'ensemble](#)
 3. [Catégorie 1 — Modèle métier](#)
 - [3.1 Classe Card](#)
 - [3.2 Classe User](#)
 - [3.3 Classe Recommendation](#)
 - [3.4 Classe Battle](#)
 - [3.5 Classe Vote](#)
 - [3.6 Classe Legitimacy](#)
 - [3.7 Classe Notification](#)
 4. [Catégorie 2 — Communication](#)
 - [4.1 Classe Request](#)
 - [4.2 Classe Response](#)
 5. [Catégorie 3 — Réseau](#)
 - [5.1 Classe NetworkClient](#)
 6. [Sérialisation JSON \(Gson\)](#)
 7. [Stockage en mémoire](#)
 8. [Cycle de vie des données](#)
 9. [Règles de cohérence](#)
 10. [Diagramme de classes](#)
 11. [Correspondance avec le cahier des charges](#)
 12. [Récapitulatif](#)
 13. [Validation](#)
-

1. Objectif

Ce document décrit l'organisation complète des données côté client de la plateforme SAE-Recommend.

Il présente :

- Les 3 catégories de données
- Chaque classe avec ses champs, types et contraintes
- Le mapping JSON ↔ Java
- Les exemples de données
- Le format de sérialisation JSON
- Le stockage en mémoire
- Le cycle de vie des données
- Les règles de cohérence

2. Vue d'ensemble

Le client organise ses données en 3 catégories.

Catégorie	Classes	Rôle
Modèle métier	Card, User, Recommendation, Battle, Vote, Legitimacy, Notification	Représenter les objets
Communication	Request, Response	Encapsuler les messages JSON
Réseau	NetworkClient	Gérer le socket TCP

3. Catégorie 1 — Modèle métier

3.1 Classe Card

Représente une carte langage de programmation.

3.1.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
id	String	Identifiant unique	Format c_XXX
title	String	Nom du langage	1-100 caractères
paradigm	String	Paradigme	OBJECT, FUNCTIONAL, PROCEDURAL, LOGIC, SCRIPTING, MULTI
description	String	Description	1-500 caractères
typing	String	Typage	STATIC, DYNAMIC, STRONG, WEAK
difficulty	String	Difficulté	BEGINNER, INTERMEDIATE, ADVANCED, EXPERT
yearCreated	int	Année de création	1950-2030
imageUrl	String	URL du logo	URL valide (optionnel)
creator	String	Créateur	Format u_XXX
owner	String	Propriétaire	Format u_XXX
status	String	Statut	ACTIVE, INACTIVE, AUTHENTICATED
value	int	VA	Entier positif
isAuthenticated	boolean	Authentifiée	true/false
authenticatedBy	String	Qui a authentifié	Format u_XXX (optionnel)

Champ	Type	Description	Contrainte
authenticatedAt	String	Quand	ISO 8601 (optionnel)
winCount	int	Victoires	Entier positif
isLegitimate	boolean	Légitime	true/false
stakeVA	int	VA engagée	Entier positif
createdAt	String	Date de création	ISO 8601 UTC

3.1.2 Représentation Java

```
public class Card {  
    private String id;  
    private String title;  
    private String paradigm;  
    private String description;  
    private String typing;  
    private String difficulty;  
    private int yearCreated;  
    private String imageUrl;  
    private String creator;  
    private String owner;  
    private String status;  
    private int value;  
    private boolean isAuthenticated;  
    private String authenticatedBy;  
    private String authenticatedAt;  
    private int winCount;  
    private boolean isLegitimate;  
    private int stakeVA;  
    private String createdAt;  
}
```

3.1.3 Mapping JSON ↔ Java

JSON	Java	Type
"id"	id	String
"title"	title	String
"paradigm"	paradigm	String
"description"	description	String
"typing"	typing	String
"difficulty"	difficulty	String
"yearCreated"	yearCreated	int

JSON	Java	Type
"imageUrl"	imageUrl	String
"creator"	creator	String
"owner"	owner	String
"status"	status	String
"value"	value	int
"isAuthenticated"	isAuthenticated	boolean
"authenticatedBy"	authenticatedBy	String
"authenticatedAt"	authenticatedAt	String
"winCount"	winCount	int
"isLegitimate"	isLegitimate	boolean
"stakeVA"	stakeVA	int
"createdAt"	createdAt	String

3.1.4 Exemple de données

```
{
  "id": "c_001",
  "title": "Python",
  "paradigm": "MULTI",
  "description": "Langage interprete, polyvalent",
  "typing": "DYNAMIC",
  "difficulty": "BEGINNER",
  "yearCreated": 1991,
  "imageUrl": "https://example.com/python.png",
  "creator": "u_001",
  "owner": "u_001",
  "status": "ACTIVE",
  "value": 15,
  "isAuthenticated": false,
  "authenticatedBy": null,
  "authenticatedAt": null,
  "winCount": 0,
  "isLegitimate": false,
  "stakeVA": 0,
  "createdAt": "2026-09-25T11:30:00Z"
}
```

3.1.5 Relations

Une carte est créée par un utilisateur.

Une carte peut recevoir plusieurs recommandations.

Une carte peut être concernée par plusieurs légitimations.

Une carte peut participer à plusieurs battles.

3.2 Classe User

Représente un utilisateur de la plateforme.

3.2.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
id	String	Identifiant unique	Format u_XXX
username	String	Pseudo	1-50 caractères
isBot	boolean	Est-ce un bot	true/false
recoCount	int	Recos émises	Entier positif
legitimacyCount	int	Légitimations obtenues	Entier positif

3.2.2 Représentation Java

```
public class User {  
    private String id;  
    private String username;  
    private boolean isBot;  
    private int recoCount;  
    private int legitimacyCount;  
}
```

3.2.3 Mapping JSON ↔ Java

JSON	Java	Type
"id"	id	String
"username"	username	String
"isBot"	isBot	boolean
"recoCount"	recoCount	int
"legitimacyCount"	legitimacyCount	int

3.2.4 Exemple de données

```
{
  "id": "u_001",
  "username": "usman",
  "isBot": false,
  "recoCount": 5,
  "legitimacyCount": 1
}
```

3.2.5 Relations

Un utilisateur peut posséder plusieurs cartes.

Un utilisateur peut émettre plusieurs recommandations.

Un utilisateur peut voter dans plusieurs battles.

3.3 Classe Recommendation

Représente une recommandation donnée par un utilisateur.

3.3.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
cardId	String	Carte recommandée	Format c_XXX
userId	String	Utilisateur qui recommande	Format u_XXX
date	String	Date de la recommandation	ISO 8601 UTC
active	boolean	Recommandation active	true/false

3.3.2 Représentation Java

```
public class Recommendation {
    private String cardId;
    private String userId;
    private String date;
    private boolean active;
}
```

3.3.3 Mapping JSON ↔ Java

JSON	Java	Type
"cardId"	cardId	String

JSON	Java	Type
"userId"	userId	String
"date"	date	String
"active"	active	boolean

3.3.4 Exemple de données

```
{
  "cardId": "c_001",
  "userId": "u_002",
  "date": "2026-09-25T11:30:00Z",
  "active": true
}
```

3.4 Classe Battle

Représente une battle entre deux cartes.

3.4.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
id	String	Identifiant unique	Format b_XXX
card1Id	String	Carte 1	Format c_XXX
card2Id	String	Carte 2	Format c_XXX
card1VA	int	VA carte 1	Entier positif
card2VA	int	VA carte 2	Entier positif
card1Votes	int	Votes carte 1	Entier positif
card2Votes	int	Votes carte 2	Entier positif
duration	int	Durée en secondes	60
endTime	String	Heure de fin	ISO 8601 UTC
status	String	Statut	EN_COURS, TERMINEE
winnerId	String	Carte gagnante	Format c_XXX (null si en cours)

3.4.2 Représentation Java

```
public class Battle {
    private String id;
```

```
private String card1Id;
private String card2Id;
private int card1VA;
private int card2VA;
private int card1Votes;
private int card2Votes;
private int duration;
private String endTime;
private String status;
private String winnerId;
}
```

3.4.3 Mapping JSON ↔ Java

JSON	Java	Type
"id"	id	String
"card1Id"	card1Id	String
"card2Id"	card2Id	String
"card1VA"	card1VA	int
"card2VA"	card2VA	int
"card1Votes"	card1Votes	int
"card2Votes"	card2Votes	int
"duration"	duration	int
"endTime"	endTime	String
"status"	status	String
"winnerId"	winnerId	String

3.4.4 Exemple de données

```
{
  "id": "b_014",
  "card1Id": "c_001",
  "card2Id": "c_005",
  "card1VA": 15,
  "card2VA": 10,
  "card1Votes": 0,
  "card2Votes": 0,
  "duration": 60,
  "endTime": "2026-09-25T11:35:00Z",
  "status": "EN_COURS",
}
```

```
"winnerId": null
}
```

3.5 Classe Vote

Représente un vote dans une battle.

3.5.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
battleId	String	Battle concernée	Format b_XXX
userId	String	Utilisateur qui vote	Format u_XXX
cardId	String	Carte choisie	Format c_XXX
weight	int	Poids du vote	Entier positif (min 1)
signature	String	Signature SHA-256	64 caractères
votedAt	String	Date du vote	ISO 8601 UTC

3.5.2 Représentation Java

```
public class Vote {
    private String battleId;
    private String userId;
    private String cardId;
    private int weight;
    private String signature;
    private String votedAt;
}
```

3.5.3 Mapping JSON ↔ Java

JSON	Java	Type
"battleId"	battleId	String
"userId"	userId	String
"cardId"	cardId	String
"weight"	weight	int
"signature"	signature	String
"votedAt"	votedAt	String

3.5.4 Exemple de données

```
{
  "battleId": "b_014",
  "userId": "u_002",
  "cardId": "c_001",
  "weight": 5,
  "signature": "a1b2c3d4e5f6...",
  "votedAt": "2026-09-25T11:32:00Z"
}
```

3.6 Classe Legitimacy

Représente une demande de légitimation.

3.6.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
id	String	Identifiant unique	Format l_XXX
cardId	String	Carte concernée	Format c_XXX
userId	String	Utilisateur demandeur	Format u_XXX
status	String	Statut	PENDING, GRANTED, REJECTED
requestedAt	String	Date de demande	ISO 8601 UTC
grantedAt	String	Date d'accord	ISO 8601 UTC (optionnel)

3.6.2 Représentation Java

```
public class Legitimacy {
    private String id;
    private String cardId;
    private String userId;
    private String status;
    private String requestedAt;
    private String grantedAt;
}
```

3.6.3 Mapping JSON ↔ Java

JSON	Java	Type
"id"	id	String

JSON	Java	Type
"cardId"	cardId	String
"userId"	userId	String
"status"	status	String
"requestedAt"	requestedAt	String
"grantedAt"	grantedAt	String

3.6.4 Exemple de données

```
{
  "id": "l_001",
  "cardId": "c_001",
  "userId": "u_002",
  "status": "PENDING",
  "requestedAt": "2026-09-25T11:40:00Z",
  "grantedAt": null
}
```

3.7 Classe Notification

Représente une notification push.

3.7.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
type	String	Type de notification	RECOMMEND, BATTLE, LEGITIMACY
from	String	Expéditeur	Format u_XXX ou username
message	String	Contenu	1-500 caractères

3.7.2 Représentation Java

```
public class Notification {
    private String type;
    private String from;
    private String message;
}
```

3.7.3 Mapping JSON ↔ Java

JSON	Java	Type
"type"	type	String
"from"	from	String
"message"	message	String

3.7.4 Exemple de données

```
{
  "type": "RECOMMEND",
  "from": "alice",
  "message": "alice a recommande votre carte Python"
}
```

4. Catégorie 2 — Communication

4.1 Classe Request

Représente une requête envoyée au serveur.

4.1.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
action	String	Nom de l'action	LOGIN, CREATE_CARD...
payload	Map	Données de l'action	Map<String, Object>

4.1.2 Représentation Java

```
public class Request {
    private String action;
    private Map<String, Object> payload = new HashMap<>();

    public Request(String action) {
        this.action = action;
    }

    public void put(String key, Object value) {
        payload.put(key, value);
    }
}
```

4.1.3 Mapping JSON ↔ Java

JSON	Java	Type
"action"	action	String
"payload"	payload	Map

4.1.4 Exemple de données

```
{
  "action": "LOGIN",
  "payload": {
    "username": "usman"
  }
}
```

4.2 Classe Response

Représente une réponse reçue du serveur.

4.2.1 Champs avec types et contraintes

Champ	Type	Description	Contrainte
status	String	Statut	OK, ERROR, NOTIFY
action	String	Nom du résultat	LOGIN_SUCCESS...
data	Map	Données de la réponse	Map<String, Object>
code	String	Code d'erreur	Si status = ERROR
message	String	Message lisible	Si status = ERROR

4.2.2 Représentation Java

```
public class Response {
    private String status;
    private String action;
    private Map<String, Object> data;
    private String code;
    private String message;

    public boolean isOk() {
        return "OK".equalsIgnoreCase(status);
    }

    public boolean isError() {
        return "ERROR".equalsIgnoreCase(status);
    }
}
```

```
}  
}
```

4.2.3 Mapping JSON ↔ Java

JSON	Java	Type
"status"	status	String
"action"	action	String
"data"	data	Map
"code"	code	String
"message"	message	String

4.2.4 Exemple de données (succès)

```
{  
  "status": "OK",  
  "action": "LOGIN_SUCCESS",  
  "data": {  
    "userId": "u_001",  
    "username": "usman"  
  }  
}
```

4.2.5 Exemple de données (erreur)

```
{  
  "status": "ERROR",  
  "code": "USERNAME_TAKEN",  
  "message": "Ce pseudo est deja utilise."  
}
```

5. Catégorie 3 — Réseau

5.1 Classe NetworkClient

Gère la communication socket avec le serveur C.

5.1.1 Champs avec types

Champ	Type	Description
socket	Socket	Connexion TCP
in	BufferedReader	Lecture
out	PrintWriter	Écriture
gson	Gson	Sérialisation
listeners	List	Listeners
running	boolean	Flag d'arrêt

5.1.2 Méthodes

Méthode	Description
connect(ip, port, username)	Ouvre le socket
send(Request)	Envoie une requête
listenLoop()	Écoute les réponses
addListener(Consumer)	Ajoute un listener
close()	Ferme le socket

5.1.3 Représentation Java

```
public class NetworkClient {  
    private Socket socket;  
    private BufferedReader in;  
    private PrintWriter out;  
    private final Gson gson = new Gson();  
    private final List<Consumer<Response>> listeners = new ArrayList<>();  
    private volatile boolean running = false;  
}
```

6. Sérialisation JSON (Gson)

6.1 Règles

- Noms de champs identiques entre Java et JSON.
- Encodage UTF-8.
- Délimiteur `\n`.

6.2 Sérialisation

```
Card card = new Card();
card.setId("c_001");
card.setTitle("Python");
card.setParadigm("MULTI");
card.setValue(15);

String json = gson.toJson(card);
```

JSON généré :

```
{
  "id": "c_001",
  "title": "Python",
  "paradigm": "MULTI",
  "value": 15
}
```

6.3 Désérialisation

```
Response resp = gson.fromJson(line, Response.class);
String userId = (String) resp.getData().get("userId");
```

7. Stockage en mémoire

7.1 Principe

Le client ne stocke rien de manière permanente.

Il utilise :

- `ObservableList<Card>` : Cartes affichées.
- `ObservableList<Battle>` : Battles.
- `ObservableList<User>` : Utilisateurs connectés.
- `Map<String, Card>` : Index par cardId.

7.2 Où sont stockées les données

Type	Où	Rôle
Cartes	CardController	Liste affichée
Battles	BattleController	Liste affichée
Utilisateurs	ProfileController	Utilisateurs connectés
Contexte	MainController	Toutes les données

7.3 Persistance

- Aucune persistance côté client.
 - Données rechargées à chaque connexion via GET_CONTEXT.
 - Si l'application est fermée, tout est perdu côté client.
-

8. Cycle de vie des données

8.1 Démarrage

Le client se lance. Aucune donnée en mémoire.

8.2 Connexion

L'utilisateur saisit IP / Port / Pseudo. Le client envoie LOGIN. Le serveur répond LOGIN_SUCCESS + CONTEXT_RESTORED. Les ObservableList sont remplies.

8.3 Actions utilisateur

L'utilisateur interagit avec l'interface. Le client envoie des requêtes. Les réponses mettent à jour les ObservableList.

8.4 Notifications push

Le serveur envoie des notifications. Le client met à jour automatiquement les listes.

8.5 Déconnexion

Le client envoie DISCONNECT. La mémoire est libérée. Le socket est fermé.

9. Règles de cohérence

- Une seule source de vérité : le serveur.
 - Aucune écriture en BDD côté client.
 - Données en lecture seule côté client.
 - Mise à jour uniquement via réponses serveur.
 - Pas de cache permanent.
 - Platform.runLater() pour l'UI.
-

10. Diagramme de classes

Le diagramme PlantUML complet est disponible dans :

- [phase1-usman-omar/diagrammes/02-diagramme-classes.puml](#)
 - [phase1-usman-omar/diagrammes/02-diagramme-classes.png](#)
-

11. Correspondance avec le cahier des charges

Exigence	Où c'est respecté
Organisation des données côté client	Ce document
Données des cartes	Classe Card
Données des recommandations	Classe Recommendation
Données des battles	Classe Battle
Données des votes	Classe Vote
Données des légitimations	Classe Legitimacy
Clients n'accèdent pas à la BDD	Section 7

12. Récapitulatif

Catégorie	Classes	Nombre
Modèle métier	Card, User, Recommendation, Battle, Vote, Legitimacy, Notification	7
Communication	Request, Response	2
Réseau	NetworkClient	1
TOTAL		10

13. Validation

Rôle	Nom	Date	Statut
Client Java	Usman	26/09/2026	✓
Client Java	Omar	26/09/2026	⌚
Blockchain + BDD	Iyore	26/09/2026	⌚